

Lab 9: Prometheus + Grafana Observability Stack with SLO Dashboards (75 min)

Learning Objectives

By the end of this lab you will be able to:

- Scrape CockroachDB's `/_status/vars` endpoint with Prometheus and confirm targets are healthy
- Identify the ~15 metrics that actually matter out of the ~2,000 CockroachDB exports
- Import the official CockroachDB Grafana dashboards and add a custom SLO view
- Write recording rules for p99 SQL latency and a burn-rate alert on an error budget
- Configure structured log channels (`OPS` , `HEALTH` , `SENSITIVE_ACCESS`) to separate files
- Capture a `debug zip` and know what is inside it before you send it to support

Prerequisites

- **Docker Desktop** (or Docker Engine) running — there is no `cockroach` binary to install
- Docker (for Prometheus + Grafana). A binary-only fallback is given in Part B.

Setup — a Cluster That Emits Metrics

`cockroach demo` picks random HTTP ports, which makes a static scrape config awkward. Use a real local cluster with fixed ports instead:

```
scripts/crdb up
```

The cluster runs in Docker (see [Lab 1](#)). From your machine it is `localhost:26257` ; from inside another container it is `crdb1:26257` . `scripts/crdb run ...` executes inside node 1.

Generate continuous load so the dashboards have something to show:

```
scripts/crdb run workload init kv --drop 'postgresql://root@crdb1:26257?sslmode=disable'

# Continuous background load so the dashboards have something to show.
# -d detaches; stop it later with: docker rm -f lab9-load
docker run -d --name lab9-load --network crdb-labs_default \
  cockroachdb/cockroach:v23.2.5 \
  workload run kv --duration=60m --concurrency=16 --read-percent=70 \
  'postgresql://root@crdb1:26257?sslmode=disable'
```

Tasks

Part A: The Metrics Endpoint (10 min)

1. Look at the raw endpoint:

```
curl -s http://localhost:8080/_status/vars | head -30
curl -s http://localhost:8080/_status/vars | wc -l
```

Roughly two thousand lines. That's the problem this lab solves.

2. Pull out the metrics that matter. These are the ones you will put on a wall:

```
curl -s http://localhost:8080/_status/vars | grep -E \
  '^(sql_service_latency_bucket|sql_conns|sql_query_count|sql_txn_abort_count|
  liveness_livenodes|ranges_underreplicated|ranges_unavailable|
  replicas_leaders_not_leaseholders|capacity_available|capacity_used|
  sys_cpu_combined_percent_normalized|rocksdb_read_amplification|
  queue_replicate_pending|txn_restarts_serializable|admission_wait_durations_kv)' \
  | grep -v '^#'
```

3. The short list, and what each one tells you:

Metric	Signal	Alert when
<code>liveness_livenodes</code>	Nodes the cluster believes are up	< expected
<code>ranges_unavailable</code>	Ranges with no quorum — data is unreadable	> 0 for 1 min
<code>ranges_underreplicated</code>	Ranges below the target replica count	> 0 for 15 min
<code>sql_service_latency_bucket</code>	Histogram for p50/p99 SQL latency	p99 over SLO
<code>sql_conns</code>	Open SQL connections	near your pool ceiling
<code>sql_query_count</code>	QPS (rate of)	for capacity trending
<code>sql_txn_abort_count</code>	Aborted transactions	rate spike
<code>txn_restarts_serializable</code>	40001 retries — contention	rising trend
<code>sys_cpu_combined_percent_normalized</code>	Per-node CPU	> 0.80 sustained
<code>capacity_available / capacity_used</code>	Disk headroom	< 20% free
<code>rocksdb_read_amplification</code>	Storage engine read cost	> 20 sustained
<code>queue_replicate_pending</code>	Backlog of replication work	sustained non-zero
<code>admission_wait_durations_kv_sum</code>	Admission control queueing (per queue: <code>_kv_</code> , <code>_elastic_cpu_</code> , <code>_sql_kv_response_</code>)	rising = overload
<code>replicas_leaders_not_leaseholders</code>	Raft leader / leaseholder split	sustained non-zero
<code>changefeed_max_behind_nanos</code>	CDC lag	over your freshness SLO

4. **Ask the same questions in SQL** — useful when you have a shell but no dashboard:

```
SELECT * FROM crdb_internal.kv_store_status LIMIT 3;
SELECT node_id, metrics->>'sql.conns' AS conns, metrics->>'sys.cpu.combined.percent-normalized' AS cpu
FROM crdb_internal.kv_node_status;
```

Part B: Prometheus (15 min)

1. **Scrape config** — `/tmp/lab9/prometheus.yml`:

```
global:
  scrape_interval: 10s
  evaluation_interval: 10s

rule_files:
  - /etc/prometheus/rules.yml

scrape_configs:
  - job_name: cockroachdb
    metrics_path: /_status/vars
    static_configs:
      - targets:
          - crdb1:8080
          - crdb2:8080
          - crdb3:8080
        labels:
          cluster: lab9
```

Scrape the nodes by their container names, on the cluster's own network. Prometheus joins `crdb-labs_default` below, so `crdb1:8080` resolves the same way on macOS, Windows and Linux. `host.docker.internal` works only on Docker Desktop, and `--network=host` works only on Linux — reaching for either is how this step becomes "works on my machine". Note the port: **8080 inside the container** for every node, not the published 8080/8081/8082 you use from your browser.

2. Recording and alerting rules — `/tmp/lab9/rules.yml`:

```

groups:
- name: crdb-slo
  interval: 10s
  rules:
    # p99 SQL service latency, in seconds
    - record: crdb:sql_latency:p99
      expr: histogram_quantile(0.99, sum(rate(sql_service_latency_bucket[1m])) by (le))

    - record: crdb:sql_latency:p50
      expr: histogram_quantile(0.50, sum(rate(sql_service_latency_bucket[1m])) by (le))

    # Fraction of statements that succeeded (our SLI)
    - record: crdb:sql_success_ratio
      expr: |
        1 - (
          sum(rate(sql_failure_count[5m]))
          /
          clamp_min(sum(rate(sql_query_count[5m])), 1)
        )

- name: crdb-alerts
  rules:
    - alert: CRDBNodeDown
      expr: (count(up{job="cockroachdb"} == 1)) < 3
      for: 1m
      labels: {severity: critical}
      annotations:
        summary: "Fewer than 3 CockroachDB nodes are up"

    - alert: CRDBUnavailableRanges
      expr: sum(ranges_unavailable) > 0
      for: 1m
      labels: {severity: critical}
      annotations:
        summary: "{{ $value }}" ranges have lost quorum"

    - alert: CRDBUnderReplicated
      expr: sum(ranges_underreplicated) > 0
      for: 15m
      labels: {severity: warning}
      annotations:
        summary: "{{ $value }}" ranges are under-replicated"

    # SL0: 99.9% of statements complete under 100 ms.
    # Fast burn: 14.4x budget burn over 1h means the 30-day budget is gone in ~2 days.
    - alert: CRDBLatencySL0FastBurn
      expr: |
        (1 - (
          sum(rate(sql_service_latency_bucket{le="0.1"}[1h]))
          / clamp_min(sum(rate(sql_service_latency_count[1h])), 1)
        )) > (14.4 * 0.001)
      for: 5m
      labels: {severity: critical}
      annotations:
        summary: "Latency error budget burning 14.4x too fast"

    - alert: CRDBHighCPU
      expr: avg by (instance) (sys_cpu_combined_percent_normalized) > 0.8

```

```

    for: 10m
    labels: {severity: warning}

- alert: CRDBDiskLow
  expr: (capacity_available / clamp_min(capacity, 1)) < 0.2
  for: 10m
  labels: {severity: warning}

```

3. Run Prometheus:

```

docker run -d --name lab9-prom -p 9090:9090 \
  --network crdb-labs_default \
  -v /tmp/lab9/prometheus.yml:/etc/prometheus/prometheus.yml \
  -v /tmp/lab9/rules.yml:/etc/prometheus/rules.yml \
  prom/prometheus

```

Prometheus is on the cluster network (so it can reach `crdb1:8080`) *and* publishes 9090 to your machine (so you can reach its UI). A container can do both.

4. **Verify the targets are up** — <http://localhost:9090/targets>. All three nodes should be `UP`. Then check the rules loaded: <http://localhost:9090/rules>.

5. Query in the Prometheus expression browser:

```

crdb:sql_latency:p99
sum(rate(sql_query_count[1m]))
sum by (instance) (sys_cpu_combined_percent_normalized)
sum(rate(txn_restarts_serializable[5m]))

```

Part C: Grafana + the SLO Dashboard (20 min)

1. Run Grafana:

```

docker run -d --name lab9-grafana -p 3000:3000 grafana/grafana

```

Log in at <http://localhost:3000> with `admin / admin`.

2. **Add the Prometheus data source** — Connections → Data sources → Prometheus → URL `http://host.docker.internal:9090` → Save & test.

3. **Import the official dashboards.** Cockroach Labs publishes dashboard JSON at <https://github.com/cockroachdb/cockroach/tree/master/monitoring/grafana-dashboards>. Import at least:

- `overview.json` — cluster health at a glance
- `sql.json` — statement throughput and latency
- `replication.json` — range and replica health
- `runtime.json` — CPU, memory, GC

4. **Build a custom SLO dashboard.** Four panels, one screen, the thing you actually put on the wall:

Panel	Query	Panel type
Availability	<code>count(up{job="cockroachdb"} == 1)</code>	Stat, threshold at 3
Throughput	<code>sum(rate(sql_query_count[1m]))</code>	Time series
Latency p50 / p99	<code>crdb:sql_latency:p50 , crdb:sql_latency:p99</code>	Time series, 2 queries
30-day error budget remaining	<code>1 - ((1 - crdb:sql_success_ratio) / 0.001)</code>	Gauge, 0–1

5. **Add a contention panel** — this is the one that catches schema problems before users do:

```
sum(rate(txn_restarts_serializable[5m]))
sum(rate(txn_restarts_writetooold[5m]))
```

A rising restart rate almost always traces back to a schema decision: a single-row counter, a sequential key, or an over-wide transaction. You saw all three in Lab 8.

6. **Prove the dashboard works.** In another terminal, create contention deliberately:

```
scripts/crdb sql -e "
CREATE TABLE IF NOT EXISTS kv.counter (name STRING PRIMARY KEY, n INT DEFAULT 0);
UPSERT INTO kv.counter VALUES ('hot', 0);"

for i in $(seq 1 20); do
  ( for j in $(seq 1 200); do
      echo "UPDATE kv.counter SET n = n + 1 WHERE name = 'hot';"
    done | scripts/crdb sql >/dev/null 2>&1 ) &
done
wait
```

Watch the restart-rate panel spike and the p99 panel follow it.

Part D: Structured Log Channels (15 min)

CockroachDB routes log events to named **channels**. In production each channel goes somewhere different: `HEALTH` to your ops alerting, `SENSITIVE_ACCESS` to the SIEM, `OPS` to the audit trail.

1. **See the default configuration:**

```
scripts/crdb run debug check-log-config
```

2. **Write a channel-split config** — `lab9/logs.yaml` in the repo root (`mkdir -p lab9` first, so the directory belongs to you rather than to Docker). The path is inside the container; the overlay mounts `./lab9` there, so the logs land on your machine where you can read them:

```

file-defaults:
  dir: /lab9/logs
  max-file-size: 10MiB
  max-group-size: 100MiB
  buffered-writes: true

sinks:
  file-groups:
    health:
      channels: [HEALTH]
      filter: INFO
    ops:
      channels: [OPS]
      filter: INFO
    security:
      channels: [SENSITIVE_ACCESS, USER_ADMIN, PRIVILEGES]
      filter: INFO
      auditable: true
    sql-exec:
      channels: [SQL_EXEC]
      filter: WARNING
    stderr:
      filter: NONE

capture-stray-errors:
  enable: true
  dir: /lab9/logs/stray

```

3. **Restart node 1 with the config.** A compose *overlay* adds the `--log-config-file` flag and mounts `./lab9` — the base file stays untouched:

```
docker compose -f docker/labs.yml -f docker/labs.logging.yml up -d crdb1
```

```

scripts/crdb sql -e "SELECT 1;"           # back up?
scripts/crdb logs 1                       # ...and reading its new config

```

Look at `docker/labs.logging.yml`: it redefines only `command` and `volumes` for `crdb1`. That is the production pattern too — one base definition, per-environment overlays, rather than editing the base file and forgetting to put it back.

4. **Generate events in each channel and find them:**

```

scripts/crdb sql -e "
  CREATE USER lab9_auditor;           -- USER_ADMIN
  GRANT SELECT ON kv.kv TO lab9_auditor; -- PRIVILEGES
  ALTER TABLE kv.kv EXPERIMENTAL_AUDIT SET READ WRITE; -- SENSITIVE_ACCESS enabled
  SELECT count(*) FROM kv.kv;         -- SENSITIVE_ACCESS event
"

ls -la lab9/logs/
grep -o '"EventType": "[^"]*" ' lab9/logs/cockroach-security*.log | sort | uniq -c

```

5. Read one structured event in full:

```
grep 'sensitive_table_access' lab9/logs/cockroach-security*.log | tail -1 | python3 -m json.tool
```

Note the fields a SIEM cares about: `Timestamp`, `EventType`, `User`, `TableName`, `Statement`, `ApplicationName`.

Part E: debug zip — What On-Call Actually Captures (10 min)

1. Capture it:

```
scripts/crdb run debug zip /tmp/debug.zip --insecure
scripts/crdb cp crdb1:/tmp/debug.zip ./debug.zip
```

2. Look inside before you hand it to anyone:

```
unzip -l ./debug.zip | head -40
unzip -l ./debug.zip | wc -l
```

3. The parts you will actually read:

Path in the zip	What it answers
<code>nodes/*/status.json</code>	Was the node live, and what was its version?
<code>nodes/*/crdb_internal.feature_usage.txt</code>	What features is this cluster using?
<code>crdb_internal.cluster_settings.txt</code>	Which settings were changed from default?
<code>nodes/*/ranges/*.json</code>	Range-level replica and lease state
<code>crdb_internal.cluster_queries.txt</code>	What was running at capture time
<code>crdb_internal.transaction_contention_events.txt</code>	Who was blocking whom
<code>nodes/*/heap.pprof</code> , <code>goroutines.txt</code>	Memory and stuck-goroutine analysis
<code>nodes/*/logs/*</code>	The logs, already collected per node

4. Redaction matters. A raw debug zip can contain query text with literal values. For anything leaving your organization:

```
scripts/crdb run debug zip /tmp/debug-redacted.zip --insecure --redact
scripts/crdb cp crdb1:/tmp/debug-redacted.zip ./debug-redacted.zip
```

Compare the sizes and spot-check that literals are gone.

5. Statement diagnostics bundle — narrower and often more useful than a full zip:

```
EXPLAIN ANALYZE (DEBUG) SELECT count(*) FROM kv.kv WHERE k > 100;
```

Download the bundle from the URL in the output, or from DB Console → SQL Activity → Statements → the statement → Diagnostics.

Cleanup

```
docker rm -f lab9-load lab9-prom lab9-grafana 2>/dev/null
scripts/crdb down
rm -rf lab9
```

Node 1 goes back to its normal configuration next time you run `scripts/crdb up`, because the logging overlay is only applied when you name it on the command line.

Lab 9 Deliverables

□ **Prometheus** scraping all three nodes with healthy targets □ **Recording rules** for p50/p99 latency and a success-ratio SLI □ **Alerts** for node loss, unavailable ranges, under-replication, error-budget burn rate □ **Grafana** with the official dashboards plus a four-panel custom SLO view □ **Contention panel** proven by generating real contention □ **Log channels** split into health / ops / security files, with a structured audit event read end to end □ **debug zip** captured, inspected, and redacted

Challenge Exercises

1. **Multi-window burn-rate alerting.** Add a second, slower burn-rate alert (6× over 6 hours) and require both windows to fire. Why does a single-window burn-rate alert page you for blips?
2. **Alert on schema-caused contention.** Write an alert that fires when `rate(txn_restarts_serializable[5m])` exceeds a threshold *and* labels it with the top contending table pulled from `crdb_internal.transaction_contention_events`.
3. **Ship SENSITIVE_ACCESS off-box.** Add an `http` sink to `logs.yaml` pointing at a local listener (`nc -l 9999`) and confirm audit events arrive over the wire.
4. **Dashboard the changefeed.** After Lab 13, add a panel for `changefeed_max_behind_nanos` and alert when CDC lag exceeds 60 seconds.

Reference

Command / metric	Purpose
<code>/_status/vars</code>	Prometheus-format metrics endpoint (per node)
<code>crdb_internal.kv_node_status</code>	Same metrics via SQL
<code>cockroach debug check-log-config</code>	Show the effective logging config
<code>--log-config-file=logs.yaml</code>	Route channels to separate sinks
<code>cockroach debug zip --redact</code>	Support bundle without query literals
<code>EXPLAIN ANALYZE (DEBUG)</code>	Per-statement diagnostics bundle
<code>histogram_quantile(0.99, ...)</code>	p99 from a <code>_bucket</code> histogram